

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - CODING GOLF COMPETITION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	C04 - Articulation (BL3)	Assessment:	Assessment Tool 1 - Coding Golf Competition
Weightage:	35% of CIA	Total Marks:	100
C04 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	M. VAISHNAV	Reg. No.:	192524251
Section / Batch:		Date:	17/08/2024

Code of Conduct

I _____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Log in to the assigned coding platform - Code chef (<https://www.codechef.com/skill-test/c-language-dsa>). Total: 100 Marks.
- You can only attempt this assessment once. Also, you will not be able to pause the assessment after starting.
- You are not allowed to switch tabs during the assessment, otherwise you will be blocked from the assessment.
- You will get a detailed report on your performance at the end of the assessment.

Section / Topic	Focus	Q Nos	Marks
Data structures	Code Chef	28	100
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Accurately interprets all problem requirements and constraints before coding.	Understands most requirements with minor omissions.	Partially understands the problem; misses some constraints.	Misinterprets the problem or ignores key requirements.
Correctness of Solution	30	Program compiles successfully and passes all hidden and visible test cases.	Program passes most test cases with minor errors.	Program passes some test cases but has logical issues.	Program fails to compile or produces incorrect results.
Data Structure & Algorithm Selection	20	Selects the most appropriate data structure and algorithm with an optimized solution.	Chooses an appropriate solution with minor inefficiencies.	Uses a workable but less efficient approach.	Uses an inappropriate data structure or algorithm.
Code Quality & Documentation	20	Well-structured, modular, readable, and properly indented code with meaningful identifiers.	Readable code with minor formatting issues.	Basic coding style with inconsistent formatting.	Poorly organized code that is difficult to understand.
Time & Space Optimization	10	Demonstrates excellent optimization and correctly analyses time and space complexity.	Good optimization with minor inefficiencies.	Limited optimization and incomplete complexity analysis.	No optimization and incorrect or missing complexity analysis.

Faculty In-charge	Course Coordinator	Program Director
Signature: _____ Date: _____	Signature: _____ Date: _____	Signature: _____ Date: _____

Statement

Consider the following C code snippet:

```

void harmonicSeries(int n) {
    double sum = 0.0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            sum += 1.0 / j;
        }
    }
    printf("Sum: %f", sum);
}
    
```

What is the time complexity of the harmonicSeries function in terms of the input size n ? (Consider all the input are included)

Select the correct answer

☐ A) $O(n)$

☒ B) $O(n^2)$

☐ C) $O(\log n)$

☐ D) $O(n \log n)$

Did you like the problem?

14 users found this helpful



Submit

Statement

Consider the following C code snippet:

```
void harmonicSeries(int n) {  
    double sum = 0.0;  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= i; j++) {  
            sum += 1.0 / j;  
        }  
    }  
    printf("Sum: %f\n", sum);  
}
```

What is the time complexity of the harmonicSeries function in terms of the input size n ? (Consider all the imports are included)

Select the correct answer

☐ A) $O(n)$

☒ B) $O(n^2)$

☐ C) $O(\log n)$

☐ D) $O(n \log n)$

Did you like the problem?

14 users found this helpful



Submit

Statement

You are given a string (S). Your task is to determine if the string can be a palindrome after deleting at most one character from it.

Input Format

- The first line contains one integer n , the size of the string - Next line contains string S .

Output Format

- Print whether S can be made palindrome after deleting at most one character.

Constraints

- $1 \leq |S| \leq 10^5$

Sample 1:

Input	Output
4 abca	true

Explanation:

We can delete b or c to make it palindrome. after deleting b the S will be aca which is palindrome.

Sample 2:

Input	Output
4 bacc	false

C

```

1 #include <stdio.h>
2 #include <stdbool.h>
3 #include <string.h>
4
5 // Function to check if the string can be a palindrome after deleting at most one character
6 bool validPalindrome(const char* s) {
7     // Write your code here
8     int l=0, r=strlen(s)-1;
9
10    while(l<r){
11        if(s[l]!=s[r]){
12            int a=l+1, b=r;
13            while(a<b){
14                if(s[a]==s[b]) a++, b--;
15                if(s[a]!=s[b]) return false;
16            }
17            a=l, b=r-1;
18            while(a<b){
19                if(s[a]==s[b]) a++, b--;
20                if(s[a]!=s[b]) return false;
21            }
22            return true;
23        }
24        l++;
25        r--;
26    }
27    return true;
28 }
29
30 int main() {
31     int n;
32     scanf("%d", &n);
33     char s[100000];
34     scanf("%s", s); // Read the input string
35     printf("%s\n", validPalindrome(s) ? "true" : "false"); // Output the result
36     return 0;
37 }
  
```

Test against Custom Input

4
abca

Statement

You are given a string (S). Your task is to determine if the string can be a palindrome after deleting at most one character from it.

Input Format

- The first line contains one integer n , the size of the string. Next line contains string S .

Output Format

- Print whether S can be made palindrome after deleting at most one character.

Constraints

- $1 \leq |S| \leq 10^5$

Sample 1:

Input	Output
4 abca	true

Explanation:

We can delete b or c to make it palindrome. after deleting b the S will be aca which is palindrome.

Sample 2:

Input	Output
4 abcb	false

```

1 #include <stdio.h>
2 #include <stdbool.h>
3 #include <string.h>
4
5 // Function to check if the string can be a palindrome after deleting at most one character
6 bool validPalindrome(const char* s) {
7     // Write your code here
8     int l=0, r=strlen(s)-1;
9
10    while(l<r){
11        if(s[l]!=s[r]){
12            int a=l+1, b=r;
13            while(s[a]==s[b]) a++, b--;
14            if(a>b) return true;
15            a=l, b=r-1;
16            while(s[a]==s[b]) a++, b--;
17            return a>b;
18        }
19        l++;
20        r--;
21    }
22    return true;
23 }
24
25 int main() {
26     int n;
27     scanf("%d", &n);
28     char s[100000];
29     scanf("%s", s); // Read the input string
30     printf("%s\n", validPalindrome(s) ? "true" : "false"); // Output the result
31 }

```

Test against Custom Input

4 abca	
-----------	--

Statement

Consider the following C program:

```
struct MinQueue {
    int queue[100], minQueue[100], front, rear, minfront, minrear;
};

void enqueue(struct MinQueue* mq, int number) {
    mq->queue[mq->rear++] = number;
    while (mq->minrear > mq->minfront && mq->minQueue[mq->minrear - 1] > number) {
        mq->minQueue[mq->minrear++] = number;
    }
}

int dequeue(struct MinQueue* mq) {
    int number = mq->queue[mq->front++];
    if (number == mq->minQueue[mq->minfront]) {
        mq->minfront++;
    }
    return number;
}

int main() {
    struct MinQueue mq = {.front = 0, .rear = 0, .minfront = 0, .minrear = 0};
    enqueue(&mq, 45);
    enqueue(&mq, 80);
    enqueue(&mq, 20);
    dequeue(&mq);
    enqueue(&mq, 5);
}
```

After executing the above operations, what is the current minimum value in the queue?

Select the correct answer

☐ 45

☒ 5

☐ 20

☐ 80

Did you like the problem?

5 users found this helpful



Solve

Statement

Consider the following C program:

```
struct MinQueue {
    int queue[100], minQueue[100], front, rear, delFront, delRear;
};

void enqueue(struct MinQueue* mq, int number) {
    mq->queue[mq->rear++] = number;
    while (mq->minRear > mq->delFront && mq->minQueue[mq->delRear] > number, mq->delRear
    == mq->minRear)
        mq->minQueue[mq->delRear++] = number;
}

int dequeue(struct MinQueue* mq) {
    int number = mq->queue[mq->delFront];
    if (number == mq->minQueue[mq->delFront]) mq->delFront++;
    return number;
}

int main() {
    struct MinQueue mq = {.front = 0, .rear = 0, .delFront = 0, .delRear = 0};
    enqueue(&mq, 4);
    enqueue(&mq, 30);
    enqueue(&mq, 20);
    dequeue(&mq);
    enqueue(&mq, 5);
}
```

After executing the above operations, what is the current minimum value in the queue?

Did you like the problem?

5 users found this helpful



Select the correct answer

☐ 4

☒ 30

☐ 20

☐ 5

Submit

Statement

Given a string `num` representing a non-negative integer and an integer `k`, remove `k` digits from `num` to form the smallest possible integer. Return this smallest possible integer as a string.

Input Format

- First line contains two integers `n` and `k`, `n` is the size of `num` and `k` is the number of digits that need to be removed.
- Second line contains a string `num`.

Output Format

- Print the smallest possible integer as a string, without leading zero except for the zero itself.

Constraints

- $1 \leq n, k \leq 100000$
- `num` will not contain any leading zero except for the zero itself.

Sample 1:

Input	Output
52 91954	124

Explanation:

Remove both the 9 and the number formed will be 124.

Sample 2:

C

```

1 #include <stdio.h>
2 #include <string.h>
3
4 // Function to remove k digits from the input number
5 char* remove_k_digits(char* num, int k) {
6     // Write your code here
7     int n = strlen(num), top = 0;
8
9     for(int i = 0; i < n; i++) {
10         while(top > 0 && num[top-1] > num[i]) {
11             num[top-1] = num[i];
12             top--;
13         }
14         num[top++] = num[i];
15     }
16
17     while(top > 0 && top < n && num[top-1] == '0') top--;
18
19     num[top] = '\0';
20
21     int len = top;
22     while(len < n && num[len] == '0') len++;
23
24     if(len < n) memmove(num, num + len, top - len + 1);
25     char* res = malloc(strlen(num) + 1);
26     strcpy(res, num);
27     return res;
28 }
29
30 int main() {
31     int n, k;
32     scanf("%d %d", &n, &k);

```

Test against Custom Input

5 2
91954

Statement

Given a string `num` representing a non-negative integer and an integer `k`, remove `k` digits from `num` to form the smallest possible integer. Return this smallest possible integer as a string.

Input Format

- First line contains two integers `n` and `k`, `n` is the size of `num` and `k` is the number of digits that need to be removed.
- Second line contains a string `num`.

Output Format

- Print the smallest possible integer as a string, without leading zero except for the zero itself.

Constraints

- $1 \leq n, k \leq 100000$
- `num` will not contain any leading zero except for the zero itself.

Sample 1:

Input	Output
52 99004	104

Explanation:

Remove both the 9 and the number formed will be 104.

Sample 2:

1080

```
1 #include <iostream>
2 #include <string.h>
3
4 // Function to remove k digits from the input number
5 char* remove_k_digits(char* num, int k) {
6     // Write your code here
7     int n = strlen(num), top=0;
8
9     for(int i=0; i<n; i++){
10         while(top>0 && num[top-1]>num[i]){
11             num[top-1]=num[i];
12             top--;
13         }
14         num[top++]=num[i];
15     }
16
17     while(top>0 && (top-->1) && num[top-1]>num[top]){
18         num[top]=num[top-1];
19     }
20
21     int i=0;
22     while(i<top && num[i]=='0') i++;
23
24     if(i==top) return num;
25     char* res = malloc(strlen(num)+1);
26     strcpy(res, num);
27     return res;
28 }
29
30 int main() {
31     int n, k;
32     scanf("%d %d", &n, &k);
```

Test against Custom Input:

5 2
99004

Statement

Consider the following C program:

```
#include <stdio.h>

int main() {
    int value1 = 5, value2 = 10;
    int *p1 = &value1, *p2 = &value2;
    int **pp1 = &p1, **pp2 = &p2;

    printf("Before modification:\n");
    printf("value1: %d, value2: %d", value1, value2);

    *p1 = *p2;
    **pp2 = 20;

    printf("After modification:\n");
    printf("value1: %d, value2: %d", value1, value2);
    printf("p1 points to: %d", *p1);
    printf("p2 points to: %d", *p2);

    return 0;
}
```

What will be the output?

Did you like the problem?

2 users found this helpful



Select the correct answer

☐ A) Before modification:
value1: 5, value2: 10
After modification:
value1: 5, value2: 20
p1 points to: 20
p2 points to: 20

☐ B) Before modification:
value1: 5, value2: 10
After modification:
value1: 20, value2: 10
p1 points to: 20
p2 points to: 10

☒ C) Before modification:
value1: 5, value2: 10
After modification:
value1: 10, value2: 20
p1 points to: 10
p2 points to: 20

☐ D) Before modification:
value1: 5, value2: 10

Submit

Statement

Consider the following C program:

```
#include <stdio.h>

int main() {
    int value1 = 5, value2 = 10;
    int *p1 = &value1, *p2 = &value2;
    int **pp1 = p1, **pp2 = p2;

    printf("Before modification:\n");
    printf("value1: %d, value2: %d\n", value1, value2);

    *p1 = *p2;
    **pp1 = 20;

    printf("After modification:\n");
    printf("value1: %d, value2: %d\n", value1, value2);
    printf("p1 points to: %d\n", *p1);
    printf("p2 points to: %d\n", *p2);

    return 0;
}
```

What will be the output?

Select the correct answer

☐ A) Before modification:
value1: 5, value2: 10
After modification:
value1: 5, value2: 20
p1 points to: 20
p2 points to: 20

☐ B) Before modification:
value1: 5, value2: 10
After modification:
value1: 20, value2: 10
p1 points to: 20
p2 points to: 10

☒ C) Before modification:
value1: 5, value2: 10
After modification:
value1: 10, value2: 20
p1 points to: 10
p2 points to: 20

☐ D) Before modification:
value1: 5, value2: 10

Did you like the problem?

2 users found this helpful



Submit

Statement

Given two integer arrays *numa1* and *numa2*, return an array of their intersection. Each element in the result must appear as many times as it shows in both arrays and return the result in sorted order.

Intersection is defined as the common element in both arrays. For example -

numa1 - [1, 2] and *numa2* - [2, 1, 3]

Therefore, only 1 and 2 are common in both arrays. On sorting the resultant array would be [1, 2].

Input Format

- The first line contains two integers *n*, *m* — the size of *numa1* and *numa2*.
- The second line contains *n* integers *a1*, *a2*, ..., *an* — the number of *numa1*.
- The third line contains *m* integers *b1*, *b2*, ..., *bm* — the number of *numa2*.

Output Format

Print all the elements that are appearing in both the arrays in sorted order.

Constraints

- $1 \leq n, m \leq 100$
- $0 \leq a_i \leq 100$
- $0 \leq b_i \leq 100$

Sample 1:

Input	Output
2 3 1 2 2 1 3	1 2

C

```

1 int* intersect(int* nums1, int nums1Size, int* nums2, int nums2Size, int* returnSize) {
2     // write your code here
3     int* intersect(int* a, int n, int* b, int m, int* r){
4         static int c[100];
5         int i, j, k = 0;
6         for(j = 0; j < n; j++)
7             for(i = 0; i < m; i++)
8                 if(a[j] == b[i]){
9                     c[k++] = a[j];
10                    b[i] = -1;
11                    break;
12                }
13            }
14        *r = k;
15        return c;
16    }
17 }
18 }
19

```

Test against Custom Input

```

2 3
1 2
2 1 3

```

Submit

Submit



Statement

Given two integer arrays `nums1` and `nums2`, return an array of their intersection. Each element in the result must appear as many times as it shows in both arrays and return the result in sorted order.

Intersection is defined as the common element in both arrays. For example -

`nums1 = [1, 2]` and `nums2 = [2, 1, 3]`

Therefore, only 1 and 2 are common in both arrays. On sorting the resultant array would be [1, 2].

Input Format

- The first line contains two integers `n`, `m` — the size of `nums1` and `nums2`.
- The second line contains `n` integers `a1, a2, ..., an` — the number of `nums1`.
- The third line contains `m` integers `b1, b2, ..., bm` — the number of `nums2`.

Output Format

Print all the elements that are appearing in both the arrays in sorted order.

Constraints

- $1 \leq n, m \leq 100$
- $0 \leq a_i \leq 100$
- $0 \leq b_i \leq 100$

Sample 1:

Input	Output
2 3 1 2 2 1 3	1 2

C

```
1 int* intersect(int* nums1, int nums1Size, int* nums2, int nums2Size, int* returnSize) {  
2     // write your code here  
3     int* intersect(int* a, int n, int* b, int m, int* r){  
4         static int c[100];  
5         int i, j, k = 0;  
6         for(j = 0; j < m; j++)  
7             for(i = 0; i < n; i++)  
8                 if(a[i] == b[j]){  
9                     c[k++] = a[i];  
10                    b[j] = -1;  
11                    break;  
12                }  
13         *r = k;  
14         return c;  
15     }  
16 }  
17  
18 }
```

Test against Custom Input

```
2 3  
1 2  
2 1 3
```

Full

Submit

Statement

Consider the following C function

```
int* nextGreaterElements(const int* nums, int numSize, int* returnSize) {  
    int* result = (int*)malloc(numSize * sizeof(int));  
    *returnSize = numSize;  
    Stack s = createStack(numSize);  
    for (int i = 0; i < numSize; ++i) {  
        while (!isEmpty(s) && nums[s->top] < nums[i % numSize]) {  
            result[s->top] = nums[i % numSize];  
            pop(s);  
        }  
        if (i < numSize) push(s, i);  
    }  
    free(s->array);  
    free(s);  
    return result;  
}
```

What is the time complexity of the given nextGreaterElements function, assuming stack default operations are already implemented? (Consider all the imports are included)

Select the correct answer

☐ A) $O(n^2)$

☒ B) $O(n)$

☐ C) $O(\log n)$

☐ D) $O(n^2 \log n)$

Did you like the problem?



Solve

```

int* nextGreaterElements(const int* nums, int numsSize, int* returnSize) {
    int* result = (int*)malloc(numsSize * sizeof(int));
    *returnSize = numsSize;
    Stack* s = createStack(numsSize);
    for (int i = 0; i < numsSize; ++i) {
        while (!isEmpty(s) && nums[s->array[s->top]] < nums[i % numsSize]) {
            result[s->array[s->top]] = nums[i % numsSize];
            pop(s);
        }
        if (i < numsSize) push(s, i);
    }
    free(s->array);
    free(s);
    return result;
}

```

What is the time complexity of the given nextGreaterElements function, assuming stack default operations are already implemented? (Consider all the imports are included)

Select the correct answer

☐ A) $O(n^2)$

☒ B) $O(n)$

☐ C) $O(n \log n)$

☐ D) $O(n^2 \log n)$



Did you like the problem?

Submit

Statement

Given an array $arr[]$ of N integers and an integer K . The task is to find the number of subarray with a maximum value equal to K .

Input Format

- The first line contains two integers N, K - the size of arr and the maximum value that should be in the subarray.
- The second line contains n integers $arr[1], arr[2], \dots, arr[n]$ - the elements of arr .

Output Format

Print the number of subarrays having K as the maximum value.

Constraints

- $1 \leq N, K \leq 10000$
- $0 \leq arr[i] \leq 10000$

Sample 1:

Input

4 3
2 1 3 4

Output

3

Explanation:

There are 3 subarrays in which the maximum element is 3, they are [2,1,3], [1,3], [3].

Sample 2:

C

```
1 #include <stdio.h>
2
3 int countSubarrays(int arr[], int n, int k) {
4     // write your code here
5     long long count = 0;
6 }
7
8 int main() {
9     int N, K;
10
11     scanf("%d", &N);
12
13     scanf("%d", &K);
14
15     int arr[N];
16
17     for (int i = 0; i < N; i++) {
18         scanf("%d", &arr[i]);
19     }
20
21     int count = countSubarrays(arr, N, K);
22
23     printf("%d", count);
24
25     return 0;
26 }
27
```

Test against Custom Input

4 3
2 1 3 4

Submit

Statement

Given an array $arr[]$ of N integers and an integer K . The task is to find the number of subarray with a maximum value equal to K .

Input Format

- The first line contains two integers N , K - the size of arr and the maximum value that should be in the subarray.
- The second line contains n integers $arr[1], arr[2], \dots, arr[n]$ - the elements of arr .

Output Format

Print the number of subarrays having K as the maximum value.

Constraints

- $1 \leq N, K \leq 10000$
- $0 \leq arr[i] \leq 10000$

Sample 1:

Input	Output
4 2 1 3 4	3

Explanation:

There are 3 subarrays in which the maximum element is 3, they are [2,1,3], [1,3], [3].

Sample 2:

C

```

1 // Write your code here
2
3 int countSubarrays(int arr[], int n, int k) {
4     // write your code here
5     long long count = 0;
6 }
7
8 int main() {
9     int N, K;
10
11     scanf("%d", &N);
12     scanf("%d", &K);
13
14     int arr[N];
15
16     for (int i = 0; i < N; i++) {
17         scanf("%d", &arr[i]);
18     }
19
20     int count = countSubarrays(arr, N, K);
21
22     printf("%d", count);
23
24     return 0;
25 }
26
27
    
```

Test against Custom Input

4 3
 2 1 3 4

Submit

Submit

Submit

Statement

You are given an integer n and m , representing the dimensions of an $n \times m$ matrix. You need to construct an $n \times m$ matrix such that the following properties are satisfied:

- Each element in the matrix is 1.
- The sum of the elements of the matrix is even.

If it is not possible then print -1.

Input Format

- The first line contains one integer n and m , the size of the matrix.

Output Format

- Print a 2d matrix with given properties or -1.

Constraints

- $1 \leq n, m \leq 100$

Sample 1:

Input	Output
2 2	<pre>1 1 1 1</pre>

Explanation:

The sum of elements of the matrix is 4 which is even.

Sample 2:

C

```

1 #include <iostream>
2
3 int main() {
4     // Write your code here
5     fcfh
6     return 0;
7 }
```

Test against Custom Input

2 2

× Close

Submit

1

Statement

You are given an integer n and m , representing the dimensions of an $n \times m$ matrix. You need to construct an $n \times m$ matrix such that the following properties are satisfied:

- Each element in the matrix is 1.
- The sum of the elements of the matrix is even.

If it is not possible then print -1 .

Input Format

- The first line contains one integer n and m , the size of the matrix.

Output Format

- Print a 2d matrix with given properties or -1 .

Constraints

- $1 \leq n, m \leq 100$

Sample 1:

Input	Output
2 2	1 1 1 1

Explanation:

The sum of elements of the matrix is 4 which is even.

Sample 2:

C

```
1 //Write your code here
2
3 int main() {
4     // Write your code here
5     return 0;
6 }
```

Test against Custom Input

2 2

Copy

Submit

Exit

Statement

What will be the value of the result, if construct_algo function is called with a value 10 (Consider all imports are included)

```
int* construct_algo(int num) {  
    int* result = (int*)malloc(num * sizeof(int));  
    if (result == NULL) {  
        exit(EXIT_FAILURE);  
    }  
  
    for (int i = 0; i < num; ++i) {  
        if (i == 0 || i == 1) result[i] = 1;  
        else result[i] = result[i-1] + result[i-2];  
    }  
  
    return result;  
}
```

Select the correct answer

☒ A) (0, 1, 1, 2, 3)

☐ B) (0, 1, 1, 2, 4)

☐ C) (0, 1, 2, 3, 5)

☐ D) (0, 1, 3, 5, 10)

Did you like the problem?

1 user found this helpful



Submit

Data structures and Algorithms in C test

59 Min : 38 Sec

30 / 30

 Total attempted

30

 Total Attempted

00

 Unattempted

Back to Assessment

Submit and Finish



Data structures and Algorithms in C test - Report

🕒 Next attempt again after 29 Days

Your Total Score

1400 / 3000 points

🟢 14 Correct Answers 🟠 16 Wrong Answers ⚪ 00 Unattempted

🔗 Share test score

🔗 View Solutions

46%

SKILL SCORE

Topic wise summary

Topic	Correct	Total	Remarks	Resources
Time complexity	1	2	🔴 Weak	Learn →
Two pointers	2	5	🔴 Weak	
Queues	3	3	🟢 Good	Learn →
Arrays	1	3	🔴 Weak	Learn →
Prefix Sum	1	3	🔴 Weak	Learn →

What will be the value of the result, if construct_algo function is called with a value 20 (Consider all imports are included)

```
int* construct_algo(int num) {  
    int* result = (int*)malloc(num * sizeof(int));  
    if (result == NULL) {  
        exit(EXIT_FAILURE);  
    }  
  
    for (int i = 0; i < num; i++) {  
        if (i % 2 == 0) result[i] = 0;  
        else result[i] = result[i - 1] + result[i - 2];  
    }  
  
    return result;  
}
```

Select the correct answer

☐ A) 0, 1, 1, 2, 3

☐ B) 0, 1, 1, 2, 4

☐ C) 0, 1, 2, 3, 5

☐ D) 0, 1, 2, 3, 10

Did you like the problem?

0 user found this helpful



Arrays	1	3	Weak	Learn →
Prefix Sum	1	3	Weak	Learn →
Stacks	1	2	Weak	Learn →
Recursion	2	4	Weak	Learn →
Matrix	1	4	Weak	Learn →
LinkedLists	1	1	Good	Learn →
Sorting	1	3	Weak	Learn →

Problem Type

Type	Correct	Total
MCQs	12	15
Coding Problems	2	15

Did you like the assessment ?



Arrays	1	3	• Weak	Learn →
Prefix Sum	1	3	• Weak	Learn →
Stacks	1	2	• Weak	Learn →
Recursion	2	4	• Weak	Learn →
Matrix	1	4	• Weak	Learn →
LinkedLists	1	1	• Good	Learn →
Sorting	1	3	• Weak	Learn →

Problem Type

Type	Correct	Total
MCQs	12	15
Coding Problems	2	15

Did you like the assessment ?

